Budget-Funnelled Monte-Carlo Tree Search

A wide-to-narrow cascade that spends a fixed simulation budget where it matters

Louay Alsakka · 2026 · *working paper / technical report*

Abstract

Monte-Carlo Tree Search (MCTS) with a policy+value network spends most of its simulation budget re-searching moves that a shallow look already reveals to be poor. We describe a simple **staged cascade**: instead of one flat search over all legal moves, a fixed budget of B simulations is run in N stages that funnel from **wide-and-shallow to narrow-and-deep** — early stages look at many candidate moves briefly and prune the obvious junk; later stages spend the remaining budget only on the handful of survivors, searched deeper and more greedily. Because expensive deep search (and the value- net evaluations it drives) is confined to a shrinking candidate set, the same nominal budget resolves in far less wall-clock per move. On a Stockfish rating ladder the cascade *appeared* to hold flat-MCTS strength at up to 4.8× less time per move — but that ladder measurement carried ±89-Elo uncertainty, and **this paper's main result is that the apparent free lunch does not survive a rigorous test.** In direct, thousands-of-games head-to-head matches we find: **at an equal simulation budget the cascade is significantly *weaker than flat MCTS (≈ −200 Elo, confidence intervals excluding zero), and at equal wall-clock it is statistically indistinguishable*** (−17 Elo, 95% CI [−66, +31]). In other words the cascade trades strength-per-simulation for speed at very close to a one-for-one rate: it lands on the *same* strength-versus-time curve as flat MCTS, with **no net efficiency gain**. We report this as a cautionary, and we think useful, negative result — a reminder that ladder-noise can manufacture an efficiency claim, and a template for testing one properly. Whether a cascade can beat flat MCTS inside a strong open-source engine (KataGo, Leela Zero/Leela Chess Zero) remains open, but our evidence on a small network says the default expectation should be *no gain*.

1. Introduction

Test-time search is the dominant lever in modern game-playing systems: given a fixed network, more simulations buy more strength [Silver 2018; Jones 2021]. But not all simulations are equally useful. In a typical position most legal moves are quickly seen to be bad; a flat MCTS nonetheless allocates exploration to them through the PUCT prior [Rosin

2011; Silver 2017], and — more importantly for wall-clock — expands and evaluates their subtrees with the value network. The expensive resource is the **network evaluation per newly expanded leaf**, and a flat search spreads it thinly across the full move set.

This paper studies a minimal way to concentrate that resource: a **budget-funnelled cascade**. The idea is old in spirit — humans and classical engines both narrow a candidate list before calculating deeply — but we phrase it as an explicit *schedule over search shape* on top of an AlphaZero-style policy+value MCTS, with the number of stages as a single free knob. Our contribution is (i) the concrete formulation, (ii) an efficiency measurement on a chess network against a calibrated Stockfish ladder, and (iii) an honest account of what it does and does not yet establish.

2. Related work

MCTS and selectivity. MCTS originates with Coulom's selectivity-and-backup formulation [Coulom 2006] and the UCT bandit analysis of Kocsis and Szepesvári [2006]; see Browne et al. [2012] for a survey. Our cascade is a search-*control* strategy layered on standard PUCT MCTS [Rosin 2011] as used in AlphaGo/AlphaZero [Silver 2016; Silver 2017; Silver 2018].

Narrowing the candidate set. Progressive strategies — progressive widening and progressive unpruning — grow the considered move set as visits accumulate [Coulom 2007; Chaslot 2008]; our cascade runs the *opposite* schedule at the root, starting wide and *shrinking* the candidate set across stages, while re-investing the freed budget in depth. Rapid Action Value Estimation shares information across moves to focus search [Gelly 2011]; playout-cap and simulation-count variation are used in KataGo for training efficiency [Wu 2019]. Classical alpha-beta engines achieve related focus via iterative deepening and candidate pre-selection; the novelty here is packaging wide→narrow funnelling as a tunable multi-stage schedule for neural MCTS.

Test-time compute. That search scales strength roughly log-linearly in simulations is documented for board games [Jones 2021] and echoed by recent inference-time-compute work in language models. The cascade targets the *efficiency* of that curve — the same strength at lower cost — rather than its ceiling.

3. Method

Let a position have legal moves `M`, a policy+value network `f`, and a total budget of `B` simulations. A cascade is a list of `N` stages, each a triple `(k_i, s_i, c_i)`:

- `k_i` — the number of candidate root moves the stage is allowed to search (the *width*);
- `s_i` — simulations spent in the stage, with `Σ s_i = B`;
- `c_i` — the PUCT exploration constant for the stage.

The candidate set starts as the top-`k_1` moves by policy prior. Stage `i` runs `s_i` PUCT simulations restricted to its candidate set; the top-`k_{i+1}` moves by visit count then pass to the next stage. Visit counts and the value cache carry across stages, so deeper stages refine — rather than restart — the estimates on survivors. After the final stage (`k_N = 1`) the most-visited move is played.

We use one monotone rule so that `N` is the only knob: widths shrink geometrically `20 → 1`, exploration falls linearly `c: 4.5 → 0.2` (broad early, greedy late), and the budget is weighted toward the later, narrower stages (`Σ s_i = B = 800`). `N = 1` recovers plain flat MCTS-800. Intuitively, the wide-shallow front cheaply discards junk, and the deep-narrow tail spends the value-net evaluations only on moves that survived scrutiny.

4. Experimental setup

- **Network.** A 3.45M-parameter convolutional policy+value network (raw single-pass strength ≈ 2262 Elo on the ladder below), deliberately small; MLX on Apple Silicon.
- **Opponents / rating.** Matches against a Stockfish `UCI_Elo` ladder (rungs 2400/2700/3000) plus a random-mover anchor; model Elo is a maximum-likelihood performance rating over the ladder. **20 games per rung**, 40 ms/move for the Stockfish side, real Lichess openings.
- **Budget.** A fixed `B = 800` simulations for every configuration, so the comparison isolates *how* the budget is spent, not *how much*.
- **Speed.** Wall-clock ms/move measured on a fixed probe set (opening, middlegame, tactical).

5. Results

N	Elo (±89)	ms/move	speedup	schedule (width×sims, wide→narrow)
1 (flat)	2683	1330	1.0×	800 sims over all moves
3	2605	903	1.5×	20→4→1
4	2683	724	1.8×	20→7→3→1
7	2605	475	2.8×	20→…→1 (7 stages)
9	2543	300	4.4×	20→…→1 (9 stages)
10	2570	275	**4.8×**	20→…→1 (10 stages)

5.1 What the ladder suggested. Rating each configuration against the Stockfish ladder, wall-clock per move falls monotonically with N — up to **4.8× faster** at N=10 (1330 → 275 ms) — while ladder Elo stays in a band (2506–2683) whose members are *statistically*

indistinguishable at ±89 (20 games/rung). Taken at face value this reads "same strength, up to 4.8× cheaper." **That reading is wrong**, and the rest of this section shows why.

5.2 Direct head-to-head at equal simulations. A *paired* match — cascade vs. flat, same network, same budget, alternating colours, played to a confidence interval — is far more sensitive to a strength *difference* than each side's absolute ladder rating. It reveals that at an equal **800-simulation** budget the cascade is **significantly weaker** than flat MCTS at every N tested:

comparison (equal 800 sims)	games	cascade score	Elo diff	95% CI
N=4 cascade vs flat	30	25.0%	**−191**	[−391, −67]
N=10 cascade vs flat	30	21.7%	**−223**	[−451, −97]

Both CIs exclude zero: the ±89 ladder had **masked a real ~200-Elo loss**. The early wide-shallow stages, with few simulations spread over a large candidate set, prune good moves that flat MCTS keeps.

5.3 Direct head-to-head at equal wall-clock — the decisive test. The cascade is *faster*, so the fair efficiency question is: at **equal thinking time**, is it stronger? We match wall-clock by giving flat MCTS the simulation count that costs the same ~275 ms as the N=10 cascade (≈165 sims) and play 200 games:

comparison (equal ~275 ms)	games	cascade score	Elo diff	95% CI
N=10 cascade (800 sims) vs flat (165 sims)	200	47.5%	**−17**	**[−66, +31]**

The interval **includes zero**: at equal wall-clock the cascade and flat MCTS are **statistically indistinguishable**. The cascade's per-simulation weakness (§5.2) and its speed (§5.1) very nearly cancel.

5.4 Conclusion of the results. The wide→narrow cascade **provides no net efficiency gain** over flat MCTS on this network: it is weaker at equal simulations and break-even at equal wall-clock, i.e. it sits on the *same* strength-versus-time curve as flat MCTS. The apparent "4.8× at equal strength" was an artifact of ±89 ladder noise, not a property of the method.

6. Limitations and the path to a rigorous result

We deliberately under-claim. Three gaps must be closed before this is a defensible efficiency result:

1. **Statistical significance.** ±89 Elo from 20 games/rung is too coarse. The clean test is a **direct head-to-head match, cascade vs. flat MCTS at an identical budget**,

over thousands of games, reporting an Elo difference with confidence intervals (and, for the efficiency claim, matched *strength* at measured *nodes/wall-clock*).
2. **One small network, one game.** Results on a 3.45M chess net may not transfer. The convincing demonstration is replication inside a **state-of-the-art open-source engine — KataGo or Leela Zero/Leela Chess Zero** — showing an improvement (or a compute saving at equal strength) against that engine's own flat MCTS.
3. **Confounds.** Wall-clock mixes cache effects and batch sizes; a fair comparison should also report value-net evaluations and node counts, and control the batched-evaluation path identically across arms.

We view the present numbers as motivation to run (1)–(2), not as a substitute for them.

7. Conclusion

Spending a fixed MCTS budget in a wide-to-narrow cascade concentrates the costly value-net evaluations on the moves that survive a cheap wide pass — an appealing idea with one interpretable knob. On a coarse rating ladder it looked like a 4.8× efficiency win. But a rigorous paired evaluation says otherwise: the cascade is significantly weaker at equal simulations and statistically indistinguishable at equal wall-clock, so on this network it delivers **no net gain** over flat MCTS. We report it as a useful negative — both because a clean null is worth recording, and because it is a concrete case of coarse measurement (±89-Elo ladder ratings) manufacturing an efficiency claim that a direct, high-N-games head-to-head dissolves. The method may still help inside a stronger engine or with a better-tuned taper; absent that evidence, the honest default is that wide→narrow funnelling buys speed and loses exactly as much strength, landing back on the flat-MCTS strength–time curve.

Acknowledgements

We thank Rémi Coulom for feedback that sharpened the scope and the evidentiary bar of this note.

References

- Browne, C. et al. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE Trans. Comput. Intell. AI Games 4(1).
- Chaslot, G. et al. (2008). *Progressive Strategies for Monte-Carlo Tree Search*. New Mathematics and Natural Computation 4(3).
- Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games 2006.

- Coulom, R. (2007). *Computing Elo Ratings of Move Patterns in the Game of Go*. ICGA Journal 30(4).
- Gelly, S., Silver, D. (2011). *Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go*. Artificial Intelligence 175(11).
- Jones, A. L. (2021). *Scaling Scaling Laws with Board Games*. arXiv:2104.03113.
- Kocsis, L., Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning*. ECML 2006.
- Rosin, C. D. (2011). *Multi-Armed Bandits with Episode Context*. Annals of Mathematics and Artificial Intelligence 61(3).
- Silver, D. et al. (2016). *Mastering the Game of Go with Deep Neural Networks and Tree Search*. Nature 529.
- Silver, D. et al. (2017). *Mastering the Game of Go without Human Knowledge*. Nature 550.
- Silver, D. et al. (2018). *A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play*. Science 362.
- Wu, D. J. (2019). *Accelerating Self-Play Learning in Go*. arXiv:1902.10565 (KataGo).

Software resources. Stockfish (https://stockfishchess.org) · Leela Chess Zero (https://lczero.org) · KataGo (https://github.com/lightvector/KataGo) · MLX (https://github.com/ml-explore/mlx). Code and the cascade sweep for this note: https://github.com/LouayAlsakka/efficient-thinking.